

Fiche de révision — Python for AI

Oct 7, 2026 · @rezan

Méthode pour le QCM

Compte environ 1 minute 30 par question pour 50 à 60 questions, et garde 10 minutes à la fin pour revenir sur les questions marquées.

1. Lis la question jusqu'au bout et repère les mots pièges : **PAS, FAUX, seulement, toujours.**
2. Pour une question avec du code, écris ce que contient chaque variable ligne par ligne avant de regarder les réponses.
3. Élimine d'abord les réponses absurdes (fonction qui n'existe pas, comme `df.sort()` ou `plt.display()`).
4. Ne bloque pas plus de 2 minutes : marque la question et passe à la suivante.
5. À la fin, ne laisse aucune case vide si les mauvaises réponses ne retirent pas de points.

Les questions reprennent souvent les phrases exactes des diapos : apprends les définitions mot pour mot.

Structures de données natives

Une structure de données est un cadre en mémoire pour stocker, organiser et gérer des données, avec un accès et une modification efficaces. Python en a 4 natives.

Structure	Exemple	Ordonnée	Mutable	À retenir
Liste (list)	<code>[1, 2, 3]</code>	Oui	Oui	Se trie sur place : <code>lst.sort()</code>
Tuple	<code>(1, 2, 3)</code>	Oui	Non	Pour réordonner, on crée un nouveau tuple : <code>tuple(sorted(t))</code>
Dictionnaire (dict)	<code>{"a": 1}</code>	Non (selon le cours)	Oui	Paires clé-valeur ; clés immuables, valeurs de n'importe quel type
Ensemble (set)	<code>{1, 2, 3}</code>	Non	Oui	Éléments uniques et immuables, pas d'index <code>s[0]</code>

Mutable = modifiable après la création (ajouter, supprimer, modifier un élément). **Non ordonné** = les éléments n'ont pas de position ni d'index fixe.

- Mutables : list, dict, set.
- Immuables : int, float, str, tuple, bool.

Exemples qui tombent souvent :

- `len({1, 2, 2, 3, 3, 3})` donne `3` (les doublons disparaissent).
- `l1.append([4, 5])` ajoute UN seul élément ; `l1.extend([4, 5])` en ajoute deux.
- `t[0] = 5` sur un tuple lève une `TypeError`.
- `{[1, 2]: "x"}` ou `{[1, 2]}` lèvent une `TypeError` : une liste ne peut être ni clé ni élément de set.
- `sorted(t)` renvoie toujours une **liste**, même si `t` est un tuple.

NumPy

NumPy est plus rapide que les listes parce que ses tableaux (ndarray) sont un seul bloc mémoire contigu avec un seul type de données.

	Listes Python	Tableaux NumPy (ndarray)
Mémoire	Pointeurs vers des objets dispersés	Un seul bloc contigu
Types	Typage dynamique : chaque élément peut avoir son type, Python vérifie chaque type à l'exécution (flexible mais lent)	Un seul type : moins flexible mais plus rapide
Surcoût	Chaque élément est un objet avec des métadonnées	Types de taille fixe sans métadonnées (<code>int32</code> = 4 octets)

Types les plus courants : int, bool, float, str, object.

On écrit	Type interne par défaut
<code>dtype=int</code>	<code>int64</code> (parfois <code>int32</code>)
<code>dtype=float</code>	<code>float64</code>
<code>dtype=bool</code>	<code>bool</code>
<code>dtype=str</code>	<code><Ux</code> , x = nombre maximal de caractères (<code>["cat", "horse"]</code> donne <code><U5</code>)

On écrit	Type interne par défaut
<code>dtype=object</code>	<code>object</code>

Un tableau ne garde qu'un type : `np.array([1, 2.5, 3])` devient `float64`, et `np.array([1, "a"])` transforme tout en texte.

Opération vectorielle = appliquée à tous les éléments en une fois, sans boucle Python explicite. Elle est plus rapide, plus lisible et plus concise.

Au lieu d'une boucle for pour...	On écrit
Additionner	<code>a + b</code>
Multiplier (élément par élément)	<code>a * b</code> : <code>[1,2,3] * [4,5,6]</code> donne <code>[4 10 18]</code>
Puissance	<code>a ** 2</code>
Appliquer une fonction	<code>np.sum(a)</code> , <code>np.sin(a)</code> , <code>np.exp(a)</code> , <code>np.sqrt(a)</code>
Moyenne	<code>np.mean(a)</code>
Filtrer	<code>a[a > 0]</code>

La boucle `for` reste utile quand une itération dépend de la précédente, ou quand NumPy ne fournit pas l'opération.

Pandas et TP1 Titanic

Pandas est une bibliothèque Python pour manipuler et analyser des données tabulaires : chargement, exploration, nettoyage, transformation, agrégation et export.

Préparation des données = transformer des données brutes en un format adapté à l'analyse ou aux modèles d'IA. 5 étapes : collecte, nettoyage, transformation, mise en forme, organisation.

Elle fait partie de la **curation des données** : gérer les données pour qu'elles restent exploitables et fiables dans le temps, sur tout leur cycle de vie (collecte, intégration, nettoyage, préparation, documentation, gestion des métadonnées, contrôle qualité, stockage, partage, préservation).

Series	DataFrame
Tableau à 1 dimension, étiqueté	Structure à 2 dimensions, étiquetée
Un seul type de données	Lignes et colonnes

Series	DataFrame
Comme une colonne de tableur	Chaque colonne est une Series

Les commandes du TP1, dans l'ordre :

Je veux...	Commande
Charger un CSV	<code>df = pd.read_csv("titanic.csv")</code>
10 premières lignes	<code>df.head(10)</code>
(lignes, colonnes)	<code>df.shape</code> donne (891, 12)
Types des colonnes	<code>df.dtypes</code> ou <code>df.info()</code>
Noms des colonnes	<code>df.columns</code>
Statistiques descriptives	<code>df.describe()</code>
Plusieurs colonnes	<code>df[["Name", "Sex", "Age"]]</code> (doubles crochets)
Filtrer	<code>df[df["Age"] > 50]</code>
Deux conditions	<code>df[(df["Sex"] == "female") & (df["Pclass"] == 1)]</code>
Manquants par colonne	<code>df.isna().sum()</code>
Lignes avec au moins un manquant	<code>df[df.isna().any(axis=1)]</code>
Lignes sans manquant	<code>df[df.notna().all(axis=1)]</code>
Supprimer les lignes avec manquants	<code>df = df.dropna()</code>
Compter les doublons	<code>df.duplicated().sum()</code>
Supprimer les doublons	<code>df = df.drop_duplicates()</code>
Renommer une colonne	<code>df = df.rename(columns={"Sex": "Gender"})</code>
Changer le type	<code>df["Pclass"] = df["Pclass"].astype("int32")</code>
Trier décroissant	<code>df.sort_values(by="Fare", ascending=False)</code>
Moyenne, médiane, écart-type	<code>.mean()</code> , <code>.median()</code> , <code>.std()</code>
Valeurs non manquantes	<code>df["Cabin"].count()</code> donne 204
Effectif par valeur	<code>df["Pclass"].value_counts()</code>
Moyenne par groupe	<code>df.groupby("Pclass")["Fare"].mean()</code>

Je veux...	Commande
Exporter sans l'index	<code>df.to_csv("titanic_clean.csv", index=False)</code>

Chiffres du Titanic à reconnaître : 891 lignes et 12 colonnes ; manquants Age 177, Cabin 687, Embarked 2 ; 183 lignes sans aucun manquant ; classes 3 : 491, 1 : 216, 2 : 184.

- `df["Age"]` est une **Series**, `df[["Age"]]` est un **DataFrame**.
- Age est en `float64` car la colonne contient des NaN, et NaN est un float.
- Les colonnes texte (Name, Sex) ont le type `object`.

Visualisation et TP2 Mall Customers

Une bonne visualisation transforme des données brutes en informations utiles. Elle sert à explorer les distributions, repérer les tendances et relations, détecter les valeurs aberrantes, comparer des groupes et communiquer les résultats.

Bibliothèque	À retenir
Matplotlib	Une des plus utilisées ; graphiques statiques, animés et interactifs ; contrôle total de la personnalisation
<code>Pyplot (import matplotlib.pyplot as plt)</code>	L'interface de tracé de Matplotlib : figures, axes et graphiques avec des commandes simples
<code>Seaborn (import seaborn as sns)</code>	Visualisation statistique, construit sur Matplotlib, travaille directement avec les DataFrames, styles et palettes attractifs

Quel graphique pour quelle question :

Question	Graphique	Commande du TP2
Effectif par catégorie	Barres / count plot	<code>df["Gender"].value_counts().plot(kind="bar")</code> ou <code>sns.countplot(data=df, x="Gender")</code>
Distribution d'une variable numérique	Histogramme	<code>plt.hist(df["Age"], bins=15, edgecolor="black")</code> ou <code>sns.histplot(...)</code>
Proportions d'un tout	Camembert	<code>.plot(kind="pie", autopct="%1.2f%%")</code>

Question	Graphique	Commande du TP2
Relation entre 2 variables numériques	Nuage de points	<code>plt.scatter(x, y)</code>
Comparer des distributions entre groupes	Boîte à moustaches	<code>sns.boxplot(data=df, x="Gender", y="Spending Score (1-100)")</code>
Idem + forme de la distribution	Violon	<code>sns.violinplot(...)</code>
Toutes les paires de variables	Pairplot	<code>sns.pairplot(df[...])</code>
Corrélations	Heatmap	<code>sns.heatmap(corr, annot=True, cmap="Blues")</code>

Personnaliser : `plt.title()`, `plt.xlabel()`, `plt.ylabel()`, `color=`, `plt.grid(True)`, puis `plt.show()` pour afficher.

- `bins` = nombre d'intervalles de l'histogramme.
- `autopct="%1.2f%%"` = pourcentage avec 2 décimales sur chaque part (56.00 % femmes, 44.00 % hommes).
- `annot=True` = la valeur écrite dans chaque case de la heatmap.
- Box plot = médiane, quartiles, valeurs aberrantes. Violon = la même chose plus la densité.
- Pairplot = nuages de points pour chaque paire, distribution de chaque variable sur la diagonale.

Interprétations du TP2 (200 clients, 5 colonnes) :

- On supprime `CustomerID` avant `corr(numeric_only=True)` : c'est un identifiant sans sens statistique.
- Corrélation proche de 0 = pas de relation **linéaire** forte, pas forcément aucun lien.
- Plus forte corrélation : Age et Spending Score, environ -0,33, donc négative et faible.
- Revenu et Spending Score : pas de relation linéaire, mais plusieurs groupes de clients distincts.
- Âge le plus représenté : 30 à 35 ans.
- Hommes et femmes : distributions de Spending Score très proches, médiane vers 50.

Programmation orientée objet (POO)

La POO est un paradigme qui organise le code autour d'objets plutôt que seulement autour de fonctions et d'instructions. Un objet regroupe des données, appelées **attributs**, et les fonctions qui agissent dessus, appelées **méthodes**.

Exemple du cours, un objet `Car` : attributs `brand`, `color`, `speed` ; méthodes `start()`, `accelerate()`, `brake()`.

Procédural	Orienté objet
Code organisé autour de fonctions	Code organisé autour d'objets
Données et fonctions généralement séparées	Données et fonctions associées regroupées
Suite d'opérations	Objets et leurs interactions
Simple pour les petits programmes	Utile pour structurer les gros programmes

La POO découpe un programme complexe en composants plus petits, réutilisables et faciles à gérer. Ses 4 avantages :

Avantage	Définition
Organisation / modularité	Le code est organisé en classes et objets indépendants
Lisibilité	La structure et les responsabilités du code sont plus faciles à comprendre
Maintenabilité	On modifie une partie du code sans toucher à tout le programme
Évolutivité / extensibilité	On ajoute de nouveaux comportements ou types d'objets sans repenser tout le programme

```
class Car:
    def __init__(self, brand, speed=0):
        self.brand = brand      # attribut
        self.speed = speed

    def accelerate(self, value): # méthode
        self.speed += value

c = Car("Renault")
c.accelerate(30)
c.accelerate(20)
print(c.speed)    # 50
```

- `__init__` = le constructeur, appelé automatiquement à la création de l'objet.
- `self` = l'objet courant (l'instance) ; chaque objet a ses propres attributs.
- `self._score` = par convention, attribut réservé à un usage interne (ce n'est pas une interdiction).

Organisation de projet

Un projet bien rangé sépare les données, les notebooks, le code source et la configuration.

```
project/
├─ data/
├─ notebooks/
├─ src/
│   ├─ preprocessing.py
│   ├─ visualization.py
│   ├─ model.py
│   └─ utils.py
├─ config.py
├─ requirements.txt
├─ README.md
└─ main.py
```

Fichier	Rôle
<code>utils.py</code>	Fonctions utilitaires générales réutilisables (logs, création de dossiers, chargement d'images...)

Fichier	Rôle
<code>config.py</code>	Paramètres du projet qu'on veut modifier facilement : <code>DATA_PATH = "data/iris.csv"</code> , <code>TEST_SIZE = 0.2</code> , <code>RANDOM_STATE = 42</code> ; on les importe avec <code>from config import DATA_PATH</code>
<code>requirements.txt</code>	Liste des packages nécessaires (<code>numpy==2.3.3</code> = version exacte), installés avec <code>pip install -r requirements.txt</code>

Commentaires	Docstrings
Expliquent pourquoi ou comment une partie du code fonctionne	Décrivent ce que fait une fonction, une classe ou un module
Commencent par <code>#</code>	S'écrivent entre triples guillemets <code>""" """</code>
Pour les personnes qui lisent le code source	Placées juste après la définition (<code>def</code> ou <code>class</code>)
Utiles pour les choix pas évidents	Accessibles par Python avec <code>help()</code>

Bonnes pratiques du cours :

- Noms de variables et de fonctions explicites.
- Commentaires et docstrings adaptés.
- Pas de code dupliqué.
- Petites fonctions ciblées.
- Constantes plutôt que valeurs écrites en dur.
- Réglages centralisés dans des fichiers de configuration.
- Environnements virtuels.
- Dépendances dans `requirements.txt`.
- Git pour le versionnage.

Pièges classiques à relire en dernier

Ces 12 points sont ceux où l'on perd le plus de points.

1. `df.dropna()`, `df.rename(...)`, `astype(...)` et `drop_duplicates()` renvoient une copie : sans `df = ...`, `df` ne change pas.
2. Filtre à deux conditions : `&` et des parenthèses, jamais `and`.

3. `df["Age"] = Series` ; `df[["Age"]]` = DataFrame.
 4. `count()` = valeurs non manquantes ; `value_counts()` = effectif de chaque valeur ; `len(df)` = nombre de lignes.
 5. `df.shape` = (lignes, colonnes), dans cet ordre.
 6. `a * b` en NumPy multiplie élément par élément, ce n'est pas un produit scalaire.
 7. Un tableau NumPy n'a qu'un type : un seul float ou un seul texte convertit tout le tableau.
 8. `sorted(t)` renvoie une liste ; un tuple n'a pas `.sort()`.
 9. Clés de dictionnaire et éléments de set : immuables. Valeurs de dictionnaire : n'importe quel type.
 10. Dictionnaire « non ordonné » : réponds comme le cours, même si Python récent garde l'ordre d'insertion.
 11. Corrélation proche de 0 = pas de lien **linéaire**, pas « aucun lien ».
 12. Seaborn est construit **sur** Matplotlib, pas l'inverse ; pyplot est l'interface de Matplotlib.
- Pour t'entraîner, refais le [QCM interactif](#) après avoir relu cette fiche.